FORMADORES { IT }

Kubernetes

4. ConfigMaps & Secrets

Índice

Qué es un ConfigMap

Cómo incorporarlo a un pod

Configurar volúmenes en un pod

Qué es un Secret

Cómo crear fácilmente estos objetos

Objetivos

- Conocer todas las posibilidades del ConfigMap
- Y del objeto Secret
- Ver cómo montarlos como volúmenes en un pod
- Ser capaces de crearlos a partir de cualquier archivo

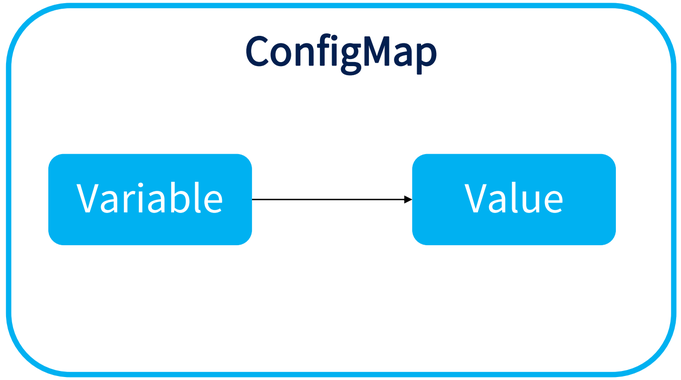
Mapa Conceptual

ConfigMap

Kubernetes también permite almacenar en el clúster todo tipo de variables y archivos de configuración.

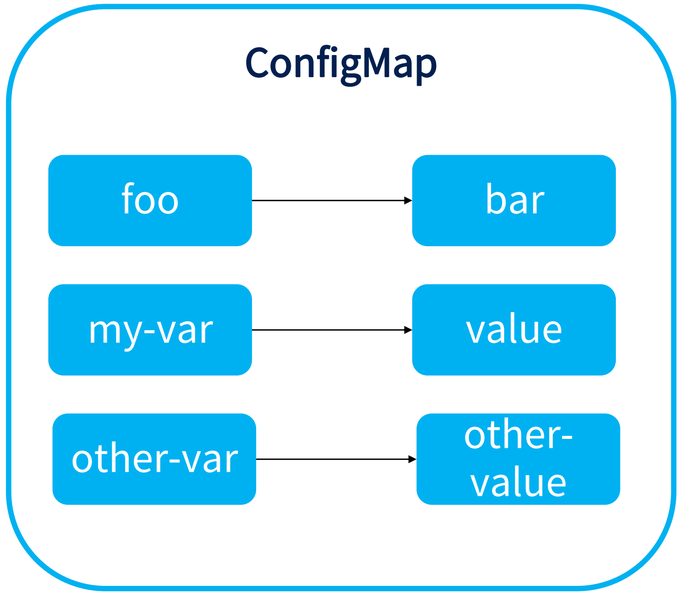
Esto lo hacemos con el objeto ConfigMap.

Un ConfigMap es un objeto muy sencillo de Kubernetes que simplemente contiene archivos de configuración y variables, que luego podemos utilizar en cualquier pod.



ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  foo: bar
  my-var: value
  other-var: other-value
```



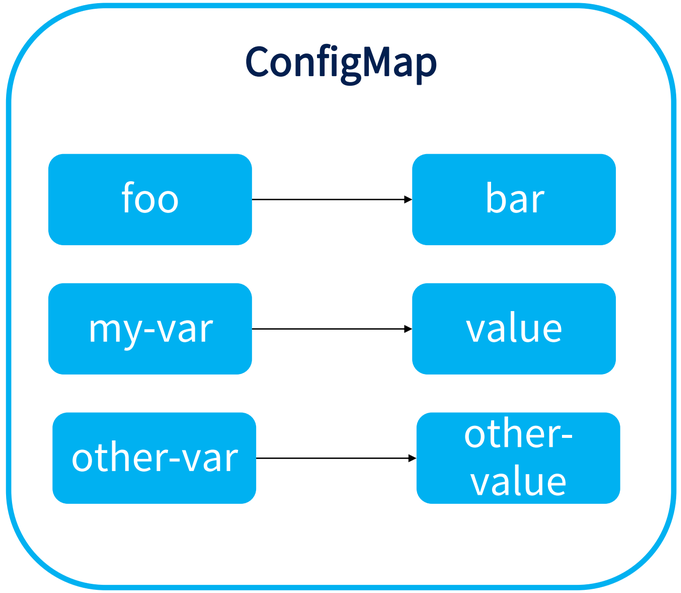
ConfigMap

También es posible definir variables con varias líneas, como archivos de configuración.

Siempre que queramos incluir varias líneas en un valor en YAML lo haremos con el símbolo |

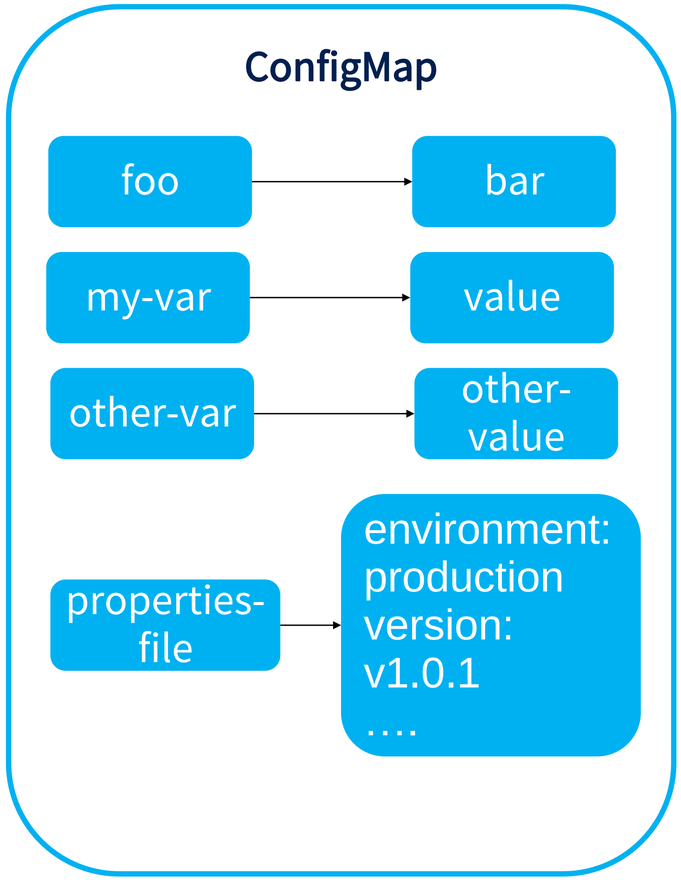
Puedes tener variables de todo tipo en el mismo ConfigMap.

El ConfigMap no está pensado para almacenar archivos grandes, no podrás almacenar más de aproximadamente 1MB de datos en él.



ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  foo: bar
  my-var: value
  other-var: other-value
  properties-file: |
    environment: production
    version: v1.0.1
    database-name: app-prod
```



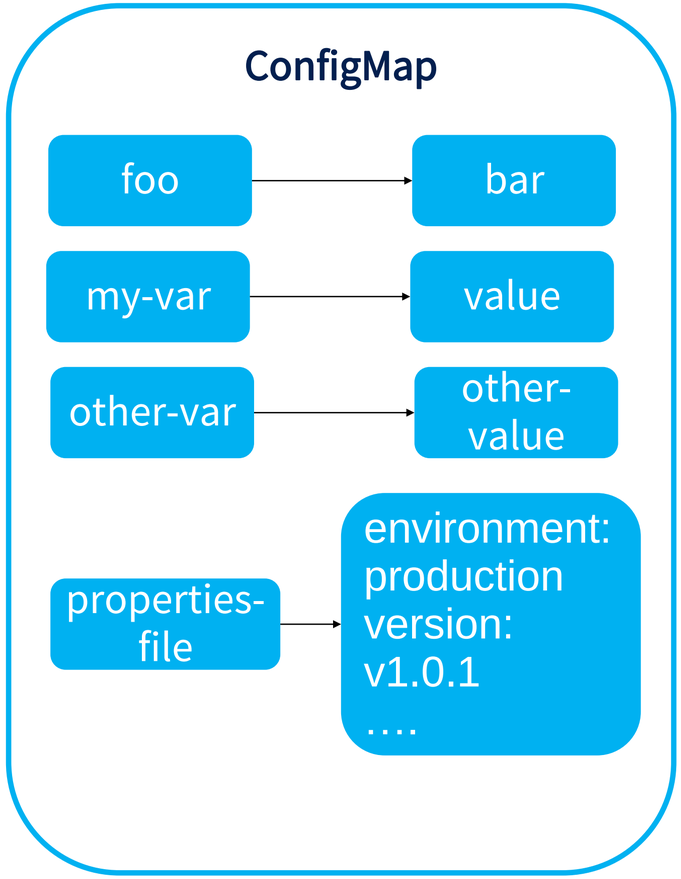
ConfigMap

Desplegamos este ConfigMap en el entorno de prueba:

https://www.katacoda.com/courses/kubernetes/playground

Recuerda pulsar el botón launch.sh para lanzar el clúster de Kubernetes.

Creamos el archivo configmap.yaml con el código anterior y lo creamos con kubectl apply -f configmap.yaml



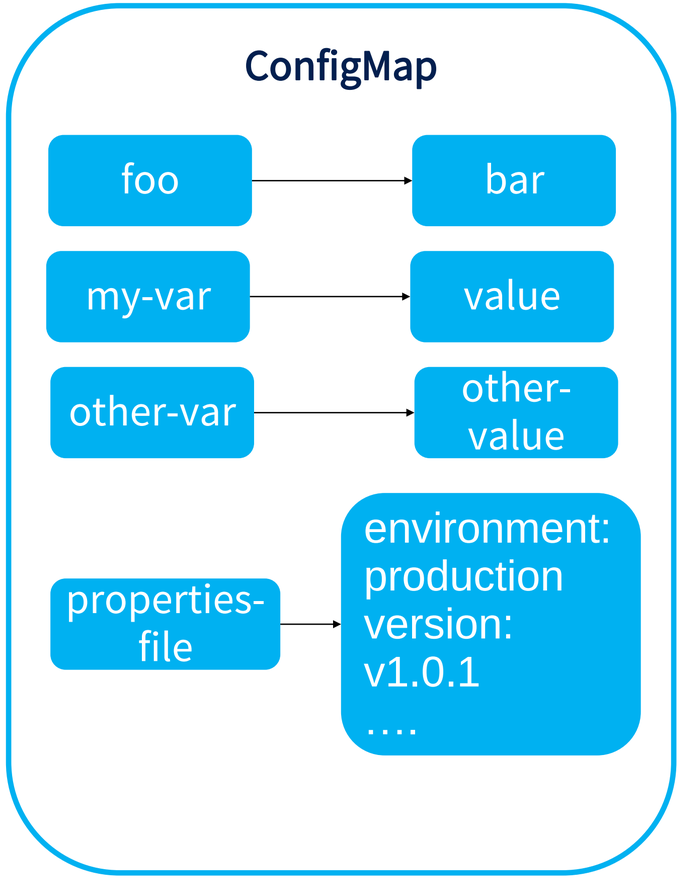
ConfigMap

Podemos listar los configmaps con el comando

kubectl get configmap

Y con el siguiente, imprimimos todos los datos que están almacenados en él:

kubectl describe configmap nginx-config

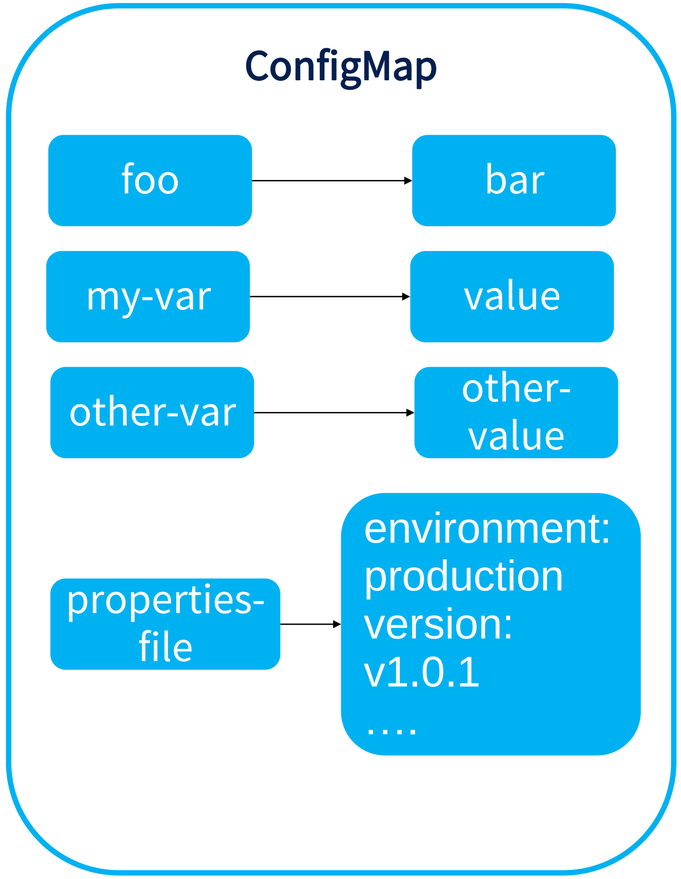


ConfigMap

¿Por qué tener mi configuración en un ConfigMap en lugar de en el propio pod?

La principal razón es que si la configuración está fuera del pod, la definición de este nos puede servir para cualquier entorno distinto.

Bastará con alterar el configmap para tener un entorno nuevo, evitando tocar la definición del pod, que sirve igualmente para todos ellos.



ConfigMap

Solo nos falta conocer cómo conectamos el ConfigMap con los pods, para que estos puedan leer la configuración.

Los ConfigMaps se montarán como directorios dentro del contenedor, en la ruta que deseemos. Dentro del directorio habrá un archivo por cada variable dentro del ConfigMap, y nuestro contenedor puede leerlos para configurar la aplicación.

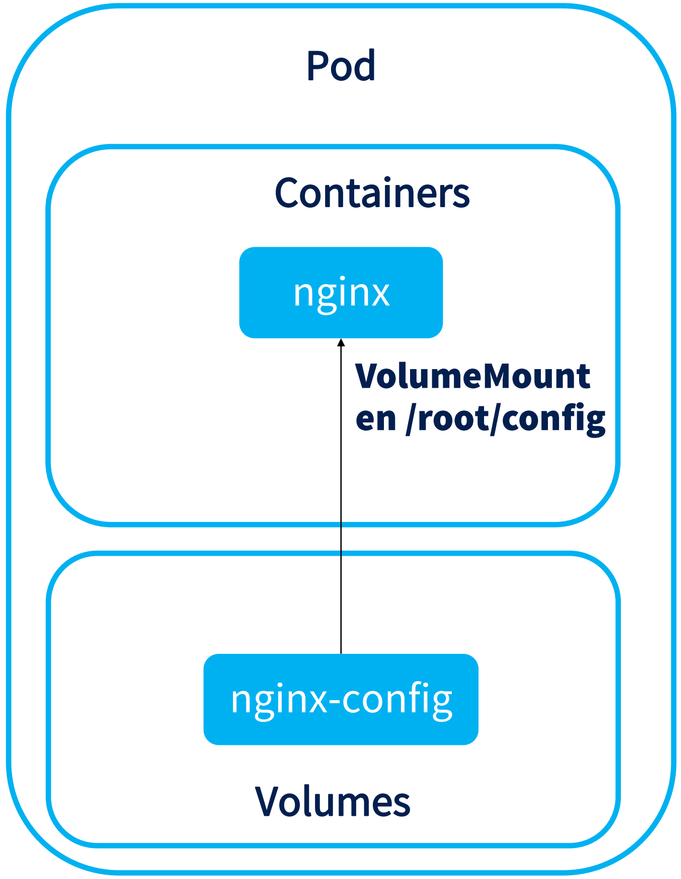
Para entender cómo montar el ConfigMap dentro del pod primero tenemos que conocer qué son los volúmenes de un pod.

Volumes

Al configurar nuestro pod podemos hacer que monte algunos directorios como discos externos o en este caso ConfigMaps con archivos de configuración.

Es igual a cuando ejecutas docker run con la opción -v

Hay distintos tipos de volúmenes que podemos montar dentro de un pod. En este módulo solo veremos los tipos configmap y secret.

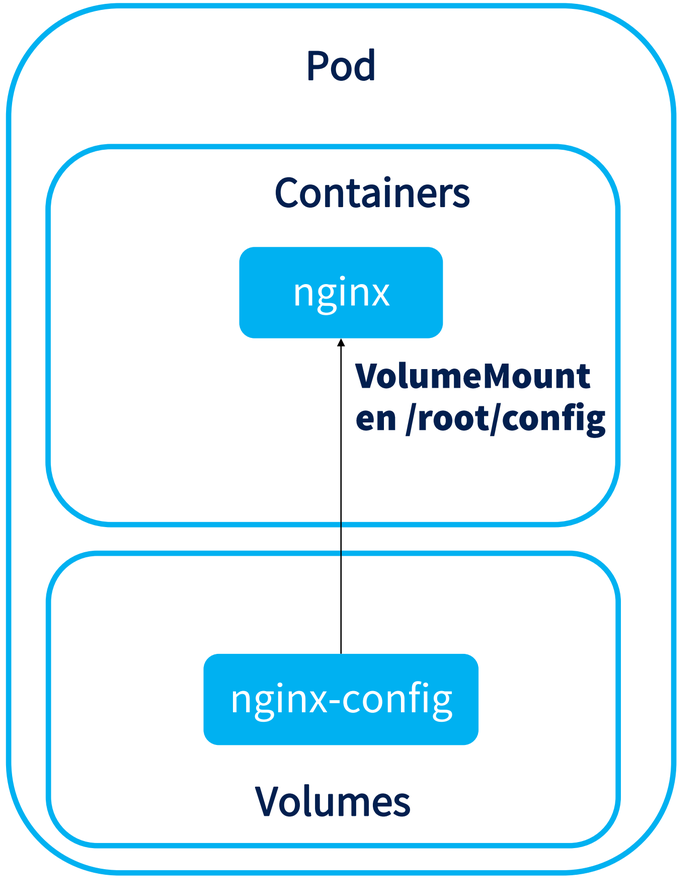


Volumes

Para añadir un volumen a un contenedor hay que hacerlo en 2 apartados del spec:

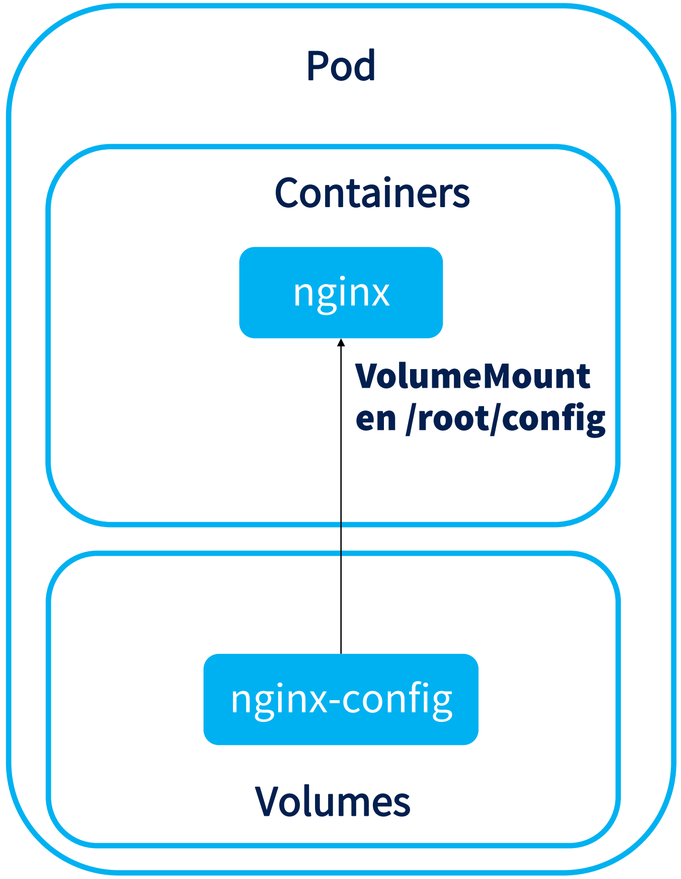
1. volumes: es una lista con todos los volúmenes que utilizaremos en el pod. Está al mismo nivel que containers.
2. volumeMounts: dentro de cada contenedor, montamos el volumen que queramos de la lista de volumes, especificando la ruta en la que lo queremos.

De esta forma podemos compartir el mismo volumen entre distintos contenedores del mismo pod por ejemplo.



Volumes

```
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: config
      mountPath: /root/config
  volumes:
  - name: config
    configMap:
      name: nginx-config
```

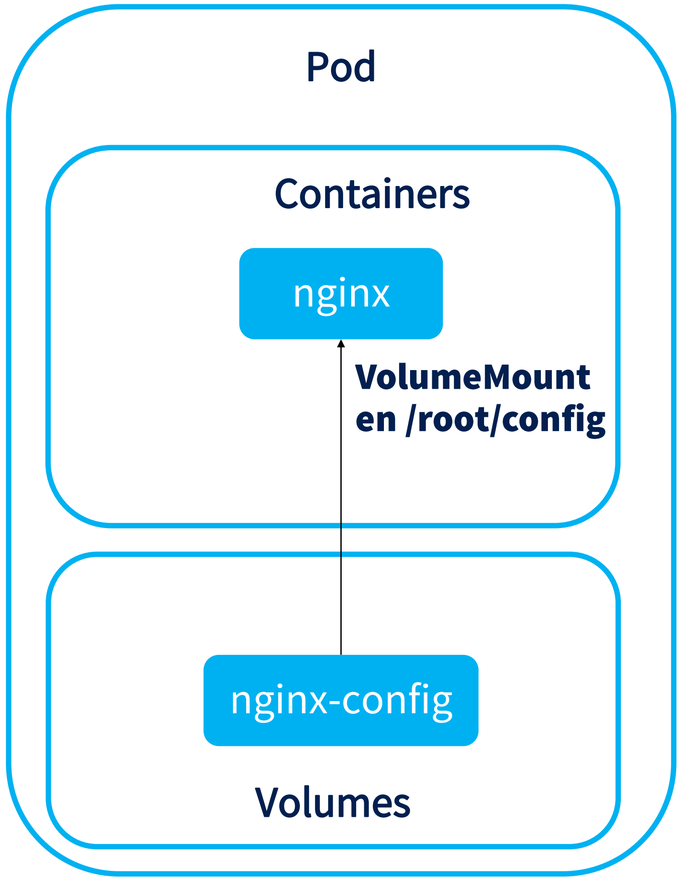


Volumes

En volumes definimos un volumen llamado config, que lee del ConfigMap llamado nginx-config. Ahora este volumen está disponible para ser montado en cualquier contenedor de ese pod.

Y en volumeMounts dentro de mi contenedor nginx utilizamos ese volumen llamado config para montarlo en la ruta /root/config.

La estructura será la misma para cualquier tipo de volumen que queramos montar.



ConfigMap

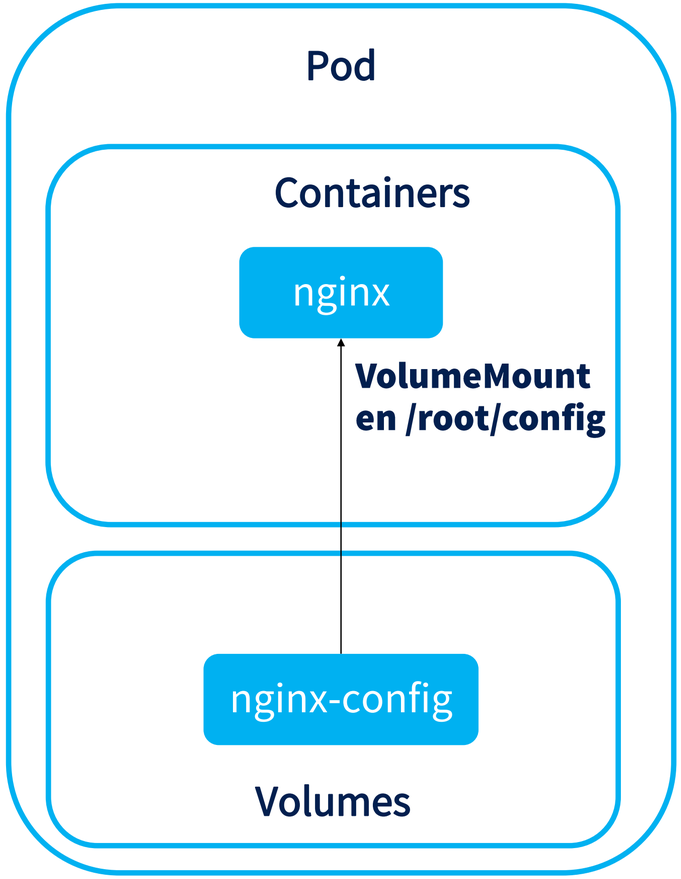
Vamos a desplegar un deployment con este volumen montado, crea un archivo deployment.yaml y aplícalo en Kubernetes

```
kubectl apply -f deployment.yaml
```

El pod de este deployment tendrá montados todos los archivos del ConfigMap en /root/config

Para verlo tenemos que conocer otro comando de kubectl, que nos va a permitir abrir un terminal dentro del pod:

```
kubectl exec<POD NAME> ls /root/config
```

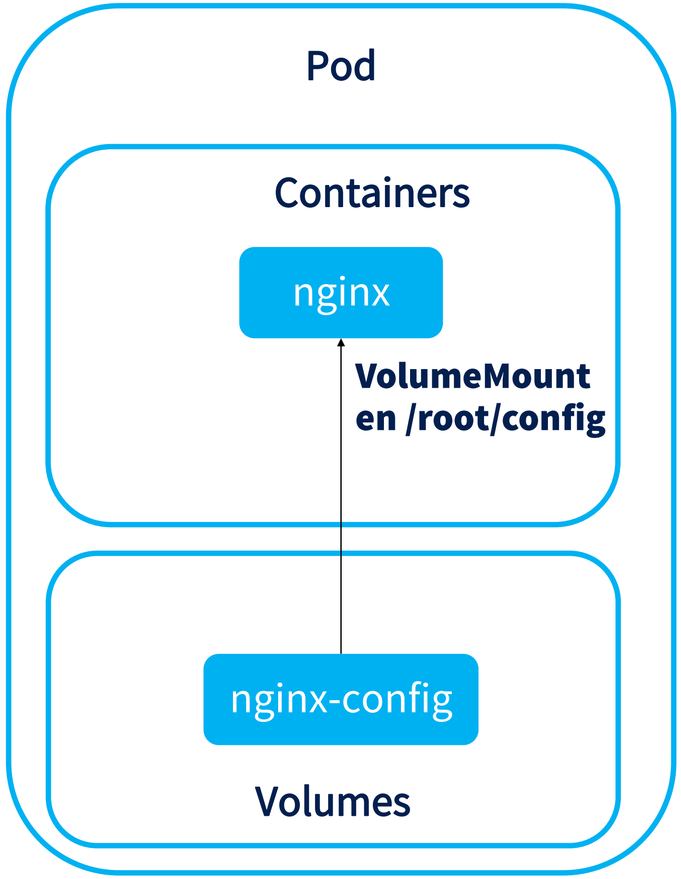


ConfigMap

El comando exec nos permite ejecutar cualquier comando dentro del pod. En este caso hemos ejecutado el comando ls /root/config para listar los archivos que se han montado desde el ConfigMap.

Podemos también ver su contenido, por ejemplo:

```
kubectl exec<POD NAME> cat /root/config/properties-file
```



ConfigMap

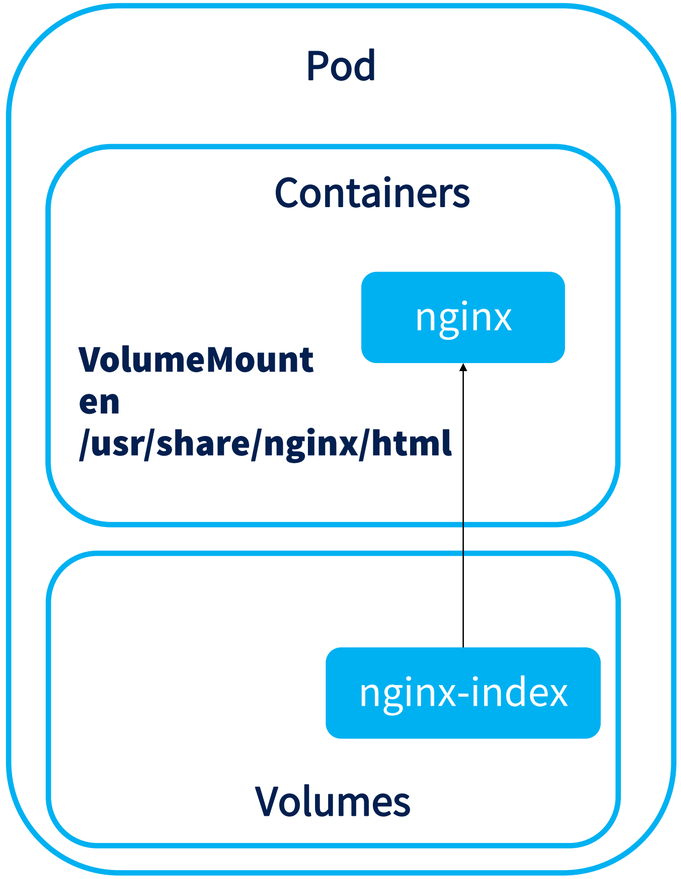
Ahora que ya conocemos cómo crear un configmap y cómo montarlo en un pod, podemos hacer algo útil con ellos, como personalizar la página de Welcome Nginx! que hemos visto hasta ahora.

Sé que la imagen de Docker que estamos desplegando, está configurada por defecto para servir las páginas que encuentre en la ruta /usr/share/nginx/html/

Por defecto tiene una página index.html en ese directorio, que es la que vemos siempre que nos conectamos con un pod de Nginx.

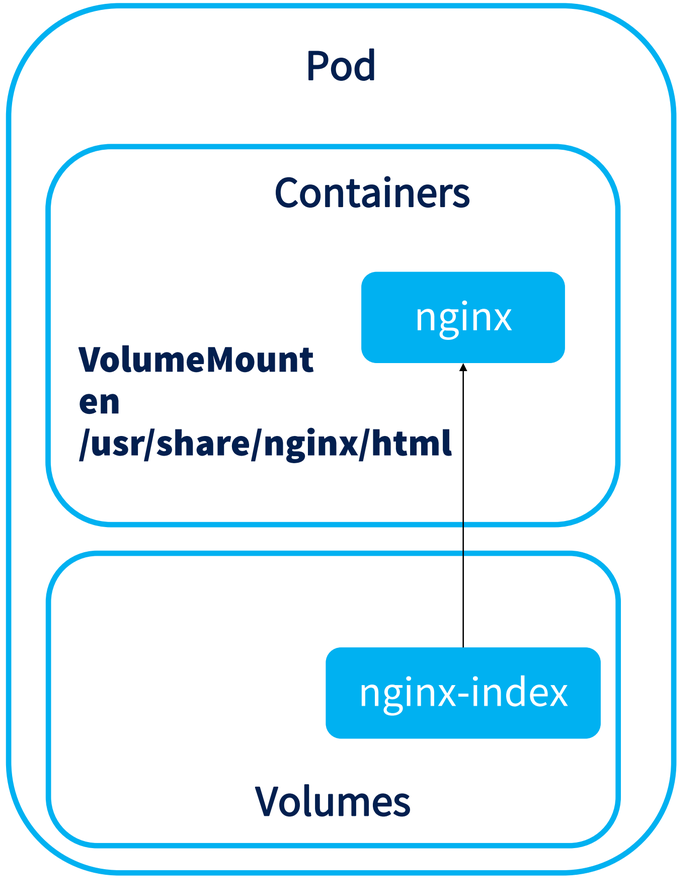
ConfigMap

Por tanto, podemos crear un configmap con un archivo index.html personalizado y montarlo en la ruta /usr/share/nginx/html/



ConfigMap

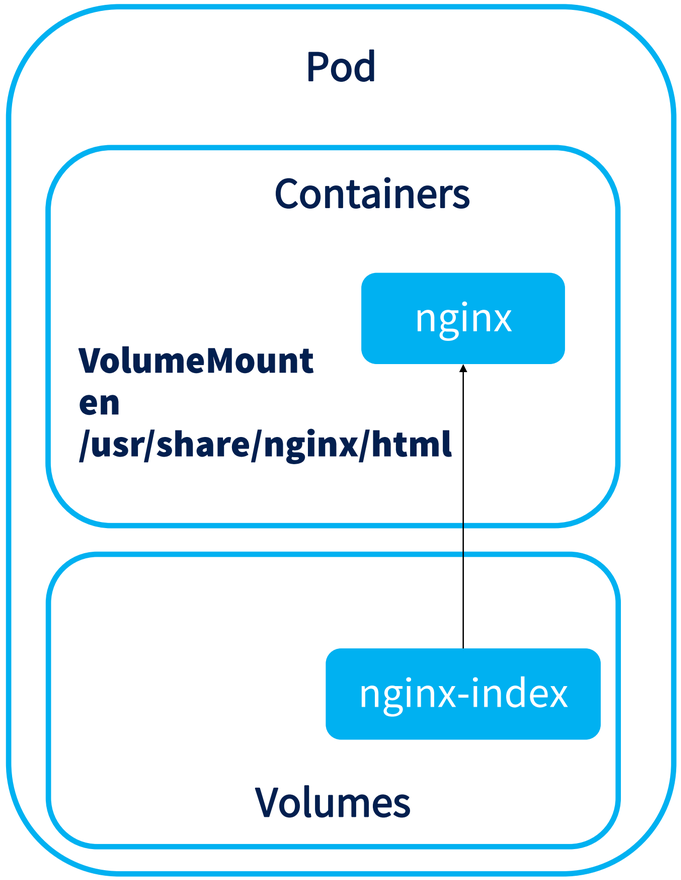
```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-index
data:
  index.html: |
    <html>
    <body>
    ¡Bienvenido a mi servidor web en Kubernetes!
    </body>
    </html>
```



ConfigMap

El spec del pod de nginx quedaría:

```
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: index
      mountPath: /usr/share/nginx/html
  volumes:
  - name: index
    configMap:
      name: nginx-index
```



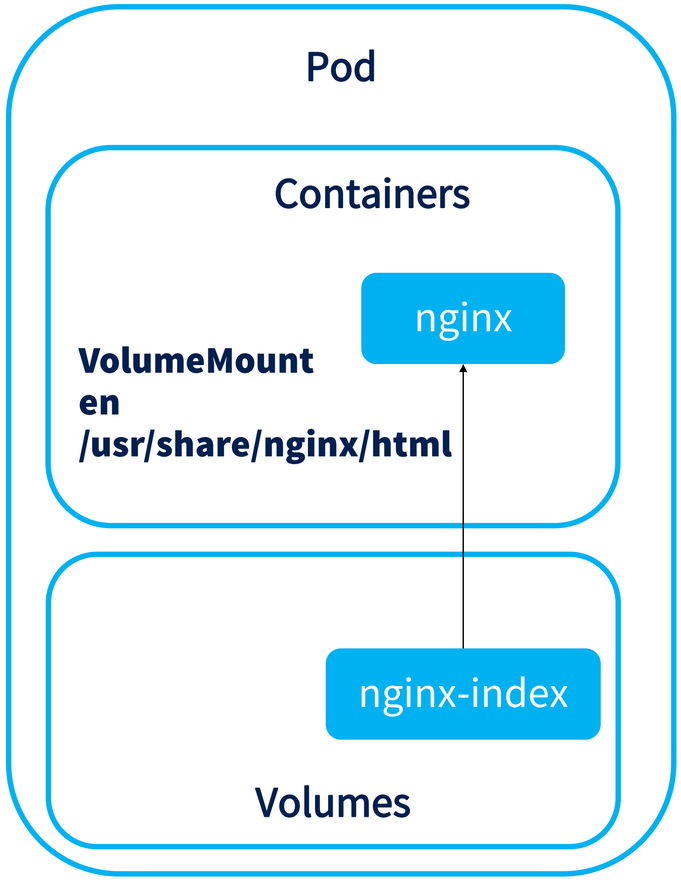
ConfigMap

Creamos el archivo nginx-index-configmap.yaml y lo aplicamos en el clúster para crear nuestro configmap.

A continuación modificamos el deployment.yaml para añadir el nuevo volumen y su correspondiente volumeMount. Aplicamos también los cambios con kubectl apply.

En este momento se creará un pod nuevo con la nueva configuración, que tendrá el archivo index.html de nuestro configmap en la ruta /usr/share/nginx/html/index.html

Accediendo con curl a la dirección IP del contenedor, ¡podremos ver nuestro servidor web personalizado!

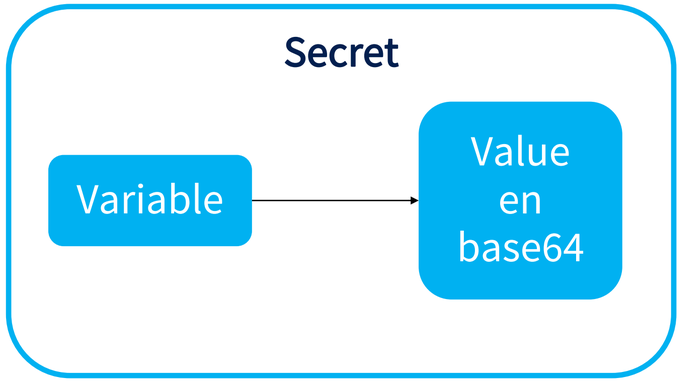


Secret

Los Secrets son exactamente iguales a los ConfigMap. Almacenan variables y archivos dentro de un objeto de Kubernetes.

La única diferencia es que el valor de cada variable en los Secrets se almacenan ofuscados en codificación base64, de forma que no se pueden leer a simple vista, antes de decodificarlos.

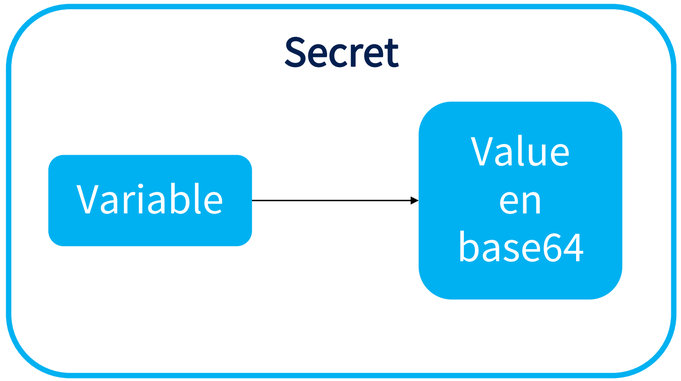
Esto hace que sea la mejor solución para almacenar datos sensibles como contraseñas o cualquier tipo de credenciales.



Secret

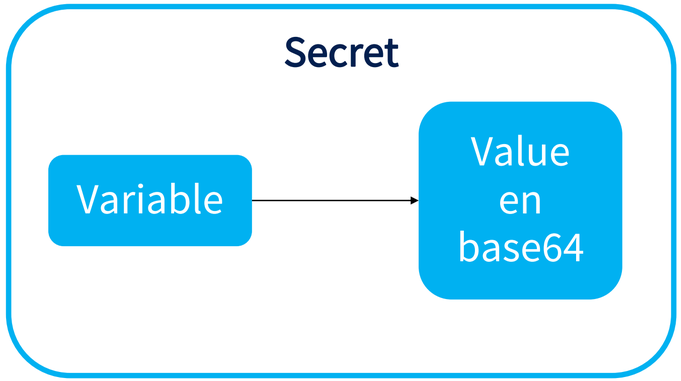
Esta sería su estructura YAML:

```
apiVersion: v1
kind: Secret
metadata:
  name: database-credentials
data:
  username: <texto en base64>
  password: <texto en base64>
```



Secret

base64 no es una forma de encriptar la información. Es una forma de codificarla. Puede ser decodificado sin necesidad de una clave. La información simplemente está ofuscada. Es como si tradujésemos nuestros secretos a otro idioma.



Para codificar en base64 utilizamos este comando:

```
echo -n 'mi-contraseña-secreta' | base64
```

Obtendremos la cadena mi-contraseña-secreta codificada en base64:

```
bWktY29udHJhc2XDsWEtc2VjcmV0YQ==
```

Secret

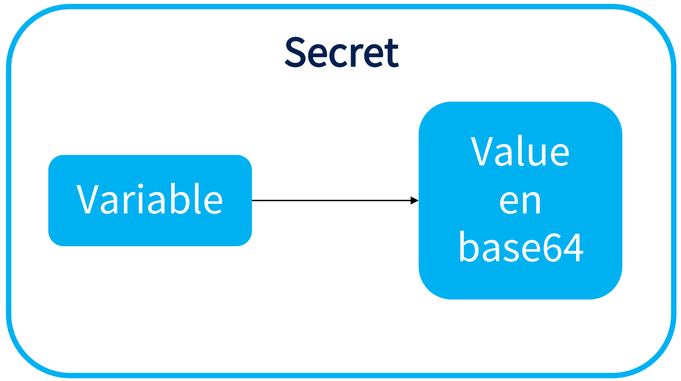
Para decodificarla sólo tenemos que hacer lo contrario:

```
echo -n 'bWktY29udHJhc2XDsWEtc2VjcmV0YQ==' | base64 -d
```

Obtendremos la cadena original:

mi-contraseña-secreta

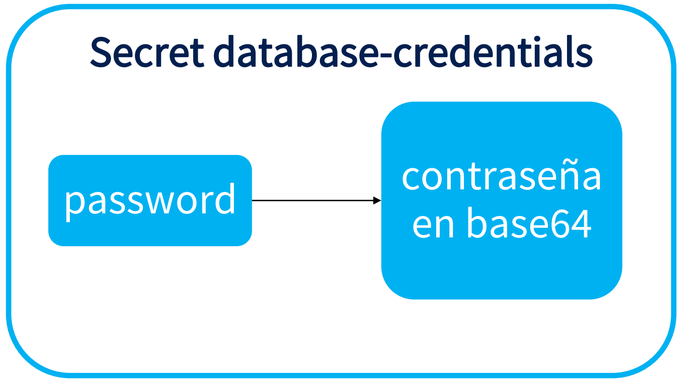
La opción -d de base64 es para decodificar



Secret

Vamos entonces a crear nuestro secret:

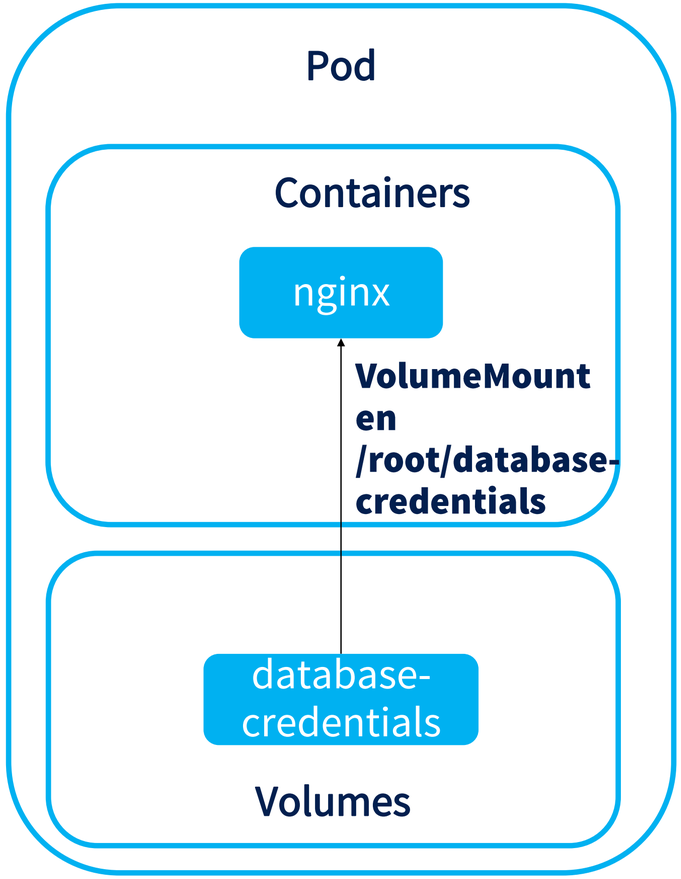
```
apiVersion: v1
kind: Secret
metadata:
  name: database-credentialas
data:
 password: 4oCYbWktY29udHJhc2XDsWEtc2VjcmV0YeKAmQ==
```



Secret

Modificamos también el spec de Nginx para que monte este secret, de la misma forma que con el ConfigMap

```
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
      - name: database-credentials
        mountPath: /root/database-credentials
  volumes:
  - name: database-credentials
    secret:
      secretName: database-credentials
```



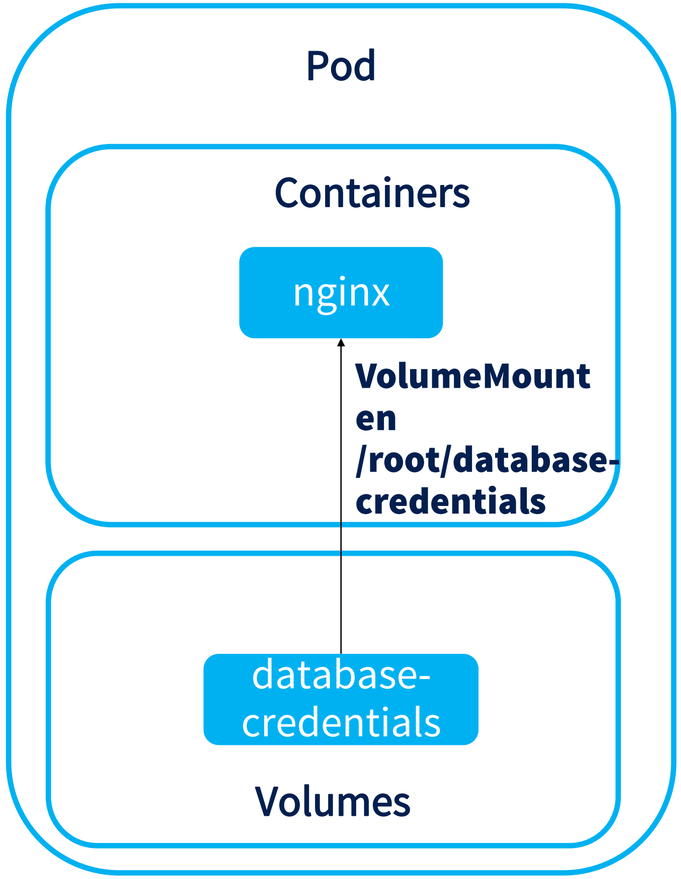
Secret

Aplicamos todos los cambios (crear el secret y modificar el deployment)

Podemos volver a utilizar kubect exec para ver el contenido del archivo

kubectl exec<POD NAME> cat /root/database-credentials/password

Vemos algo muy importante, y es que cuando se montan los archivos del secret en los pods, estos aparecen ya **decodificados**. Kubernetes los decodifica por nosotros, asi que nuestra app sólo necesita leer los archivos.



Creando Secrets desde archivos

Crear secrets codificando nosotros a mano cada variable sería una tarea tediosa para simplemente almacenar unas cuantas claves en Kubernetes.

Por eso kubectl nos da algunos comandos para automatizar la creación de secrets.

Por ejemplo:

```
kubectl create secret generic new-database-credentials --from-literal password=mi-contraseña-secreta
```

Creando Secrets desde archivos

--from-literal nos permite pasarle el conjunto clave=valor que queremos en nuestro secret. Podemos añadir este parámetro varias veces para crear distintas claves en el mismo secret

El termino generic es porque estamos creando un secret del tipo generic. Hay otros tipos de secrets que usa Kubernetes, que no nos interesan de momento.

Podemos inspeccionar el secret que acabamos de crear:

```
kubectl describe secret new-database-credentials
```

Creando Secrets desde archivos

También es posible crear secretas a partir del contenido de archivos.

Si tenemos un archivo password.txt, podemos crear un secret con un elemento que tenga como clave el nombre del archivo (password.txt) y como valor, su contenido.

```
kubectl create secret generic database-credentials-file --from-file ./password.txt
```

```
kubectl describe secret database-credentials-file
```

Creando Secrets desde archivos

El comando kubectl créate secret ofrece muchas posibilidades más, como crear un secret con todos los archivos que haya en una carpeta, o personalizar las claves de los elementos.

Puedes ver todas las posibilidades en la ayuda de kubectl:

```
kubectl create secret generic -h
```

Creando ConfigMaps desde archivos

Exactamente los mismos parámetros nos sirven para crear configmaps en un comando y desde archivos. Por ejemplo:

```
kubectl create confimap my-properties --from-literal
environment=production
```

```
kubectl create confimap my-properties-new --from-file
./production-properties.txt
```

¡Fin del módulo!

En este módulo hemos aprendido a almacenar la configuración de nuestras aplicaciones en Kubernetes, para que los pods puedan leerlas y configurar la aplicación correctamente.

- Cómo crear un ConfigMap con variables y archivos
- Cómo montarlo como un volumen dentro del pod
- Hemo montado una página HTML para nuestro servidor de Nginx desde un ConfigMap
- Aprendido a crear secrets, y sus ventajas frente a ConfigMap
- Utilizado kubectl para crear estos recursos de forma más sencilla

Extras

Páginas de la documentación oficial:

ConfigMap
Secrets

¡Gracias!